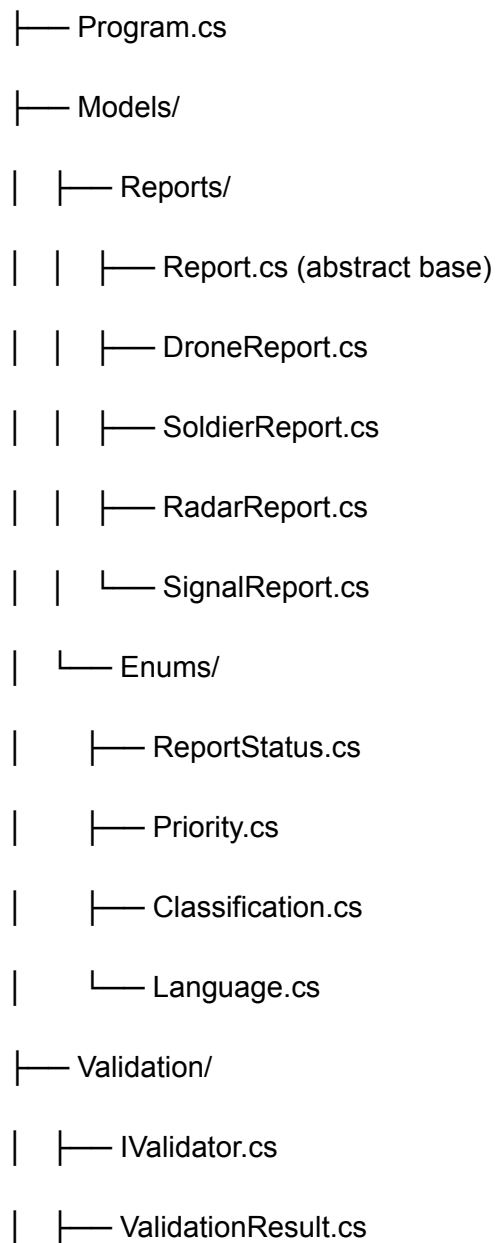


Intelligence Pipeline System - Reference Architecture

Project Structure

IntelligencePipeline/



```
| | — BaseValidator.cs
| | — DroneValidator.cs
| | — SoldierValidator.cs
| | — RadarValidator.cs
| | — SignalValidator.cs
| — Calculators/
| | — ReliabilityCalculator.cs
| | — PriorityCalculator.cs
| | — ClassificationCalculator.cs
| — Storage/
| | — ReportRepository.cs
| | — RejectedReportRepository.cs
| — Pipeline/
|   — ReportPipeline.cs
```

Namespaces

- `IntelligencePipeline` - Program entry point
 - `IntelligencePipeline.Models.Reports` - Report hierarchy
 - `IntelligencePipeline.Models.Enums` - Enumerations
 - `IntelligencePipeline.Validation` - Validation components
 - `IntelligencePipeline.Calculators` - Calculation logic
 - `IntelligencePipeline.Storage` - Data storage
 - `IntelligencePipeline.Pipeline` - Processing pipeline
-

Enumerations

ReportStatus

public enum ReportStatus

```
{  
  
    New,  
  
    Validating,  
  
    Validated,  
  
    Rejected,  
  
    InProgress,  
  
    Completed  
}
```

Represents the current state of a report in the processing workflow.

Priority

public enum Priority

```
{  
  
    Low,  
  
    Medium,  
  
    High,  
  
    Critical  
}
```

Represents the urgency level of a report.

Classification

public enum Classification

```
{  
  
    Unclassified,  
  
    Restricted,  
  
    Secret,  
  
    TopSecret  
}
```

Represents the security classification level.

Language

public enum Language

```
{  
  
    Hebrew,  
  
    Arabic,  
  
    English,  
  
    Russian,  
  
    Other  
}
```

Represents the language of signal intelligence content.

Abstract Base Class: Report

Purpose

Models the shared concept of an intelligence report regardless of source type.

Responsibility

- Holds all common report data
- Provides abstract contract for source-specific behavior
- Manages report lifecycle state
- **Does NOT enforce validation** - validation is handled by Validator classes

Properties (Public - Auto-Properties)

```
public int ReportId { get; protected set; }
```

```
public DateTime Timestamp { get; protected set; }
```

```
public double Latitude { get; protected set; }
```

```
public double Longitude { get; protected set; }
```

```
public string Description { get; protected set; }
```

```
public ReportStatus Status { get; set; }
```

```
public Priority Priority { get; set; }
```

```
public Classification Classification { get; set; }
```

```
public int ReliabilityScore { get; set; }
```

```
public string RejectionReason { get; set; }
```

Important Design Decision:

- Core data properties use **protected set** accessor - can only be modified within the Report class or its derived classes
- This provides **security** (external code cannot modify validated data) while maintaining **flexibility** (derived classes can update if needed)

- **No validation logic in properties** - all validation is performed by Validator classes (BaseValidator and its derived classes)
- Status and calculated fields use **set** accessor - can be modified during processing
- This design allows rejected reports to be stored with their original invalid values for audit and debugging purposes

Constructor (Protected)

```
protected Report(int reportId, DateTime timestamp, double latitude,
                double longitude, string description)
```

- Visibility: **protected** (only derived classes can call)
- Initializes all common fields
- Sets **Status = ReportStatus.New**
- **Does NOT validate** - simply assigns values to properties

Abstract Methods

```
public abstract string GetSourceType();
```

```
public abstract int CalculateReliabilityScore();
```

Virtual Methods

```
public virtual string GetSummary()
```

Returns a formatted summary of the report. Derived classes can override for source-specific formatting.

Override Methods

```
public override string ToString()
```

Returns a complete string representation of the report.

Relationships

- **Inherited by:** DroneReport, SoldierReport, RadarReport, SignalReport
 - **Used by:** ReportPipeline, ReportRepository, RejectedReportRepository
-

Derived Class: DroneReport

Purpose

Represents an intelligence report from a drone source.

Responsibility

- Stores drone-specific data (Altitude, ImageQuality)
- Calculates reliability based on drone criteria
- **Does NOT validate** - validation is handled by DroneValidator

Properties (Public - Auto-Properties)

```
public int Altitude { get; protected set; }
```

```
public int ImageQuality { get; protected set; }
```

Protected properties - use `protected set` to allow modification only within class hierarchy

Constructor

```
public DroneReport(int reportId, DateTime timestamp, double latitude,
```

```
    double longitude, string description,
```

```
    int altitude, int imageQuality)
```

```
: base(reportId, timestamp, latitude, longitude, description)
```

- Calls base constructor with common fields
- Initializes drone-specific fields
- **Does NOT validate** - simply assigns values

Override Methods

```
public override string GetSourceType() => "Drone";
```

```
public override int CalculateReliabilityScore()
```

Implements drone-specific reliability calculation:

- Base: 5

- ImageQuality >= 80: +3
- ImageQuality >= 50: +2
- Altitude 500–3000: +2
- Altitude > 7000: -2
- Result clamped to 1–10

Relationships

- **Inherits from:** Report
 - **Validated by:** DroneValidator
-

Derived Class: SoldierReport

Purpose

Represents an intelligence report from a soldier in the field.

Responsibility

- Stores soldier-specific data (SoldierName, SoldierID, Unit, ConfidenceLevel)
- Calculates reliability based on soldier criteria
- **Does NOT validate** - validation is handled by SoldierValidator

Properties (Public - Auto-Properties)

```
public string SoldierName { get; protected set; }
```

```
public string SoldierID { get; protected set; }
```

```
public string Unit { get; protected set; }
```

```
public int ConfidenceLevel { get; protected set; }
```

Protected properties - use **protected set** to allow modification only within class hierarchy

Constructor

```
public SoldierReport(int reportId, DateTime timestamp, double latitude,
    double longitude, string description,
    string soldierName, string soldierID, string unit,
```


int confidenceLevel)

: base(reportId, timestamp, latitude, longitude, description)

Override Methods

public override string GetSourceType() => "Soldier";

public override int CalculateReliabilityScore()

Implements soldier-specific reliability calculation:

- Base: 4
- Add ConfidenceLevel value
- Description contains weapon/vehicle/movement/explosion: +1
- Result clamped to 1–10

Relationships

- **Inherits from:** Report
 - **Validated by:** SoldierValidator
-

Derived Class: RadarReport

Purpose

Represents an intelligence report from a radar system.

Responsibility

- Stores radar-specific data (Speed, Direction, Distance)
- Calculates reliability based on radar criteria
- **Does NOT validate** - validation is handled by RadarValidator

Properties (Public - Auto-Properties)

public int Speed { get; protected set; }

public int Direction { get; protected set; }

public int Distance { get; protected set; }

Protected properties - use `protected set` to allow modification only within class hierarchy

Constructor

```
public RadarReport(int reportId, DateTime timestamp, double latitude,  
  
    double longitude, string description,  
  
    int speed, int direction, int distance)  
  
: base(reportId, timestamp, latitude, longitude, description)
```

Override Methods

```
public override string GetSourceType() => "Radar";  
  
public override int CalculateReliabilityScore()
```

Implements radar-specific reliability calculation:

- Base: 6
- Distance 500–30000: +2
- Speed 10–900: +1
- Distance > 70000: -2
- Speed > 1500: -2
- Result clamped to 1–10

Relationships

- **Inherits from:** Report
 - **Validated by:** RadarValidator
-

Derived Class: SignalReport

Purpose

Represents an intelligence report from a signal intelligence system.

Responsibility

- Stores signal-specific data (Frequency, Content, Language, SignalStrength)

- Calculates reliability based on signal criteria
- **Does NOT validate** - validation is handled by SignalValidator

Properties (Public - Auto-Properties)

```
public double Frequency { get; protected set; }
```

```
public string Content { get; protected set; }
```

```
public Language Language { get; protected set; }
```

```
public int SignalStrength { get; protected set; }
```

Protected properties - use `protected set` to allow modification only within class hierarchy

Constructor

```
public SignalReport(int reportId, DateTime timestamp, double latitude,
    double longitude, string description,
    double frequency, string content, Language language,
    int signalStrength)
```

```
: base(reportId, timestamp, latitude, longitude, description)
```

Override Methods

```
public override string GetSourceType() => "Signal";
```

```
public override int CalculateReliabilityScore()
```

Implements signal-specific reliability calculation:

- Base: 5
- SignalStrength >= -40: +3
- SignalStrength >= -70: +2
- Content contains attack/target/border/vehicle: +1
- SignalStrength < -100: -2
- Result clamped to 1–10

Relationships

- **Inherits from:** Report
 - **Validated by:** SignalValidator
-

Interface: IValidator

Purpose

Defines the contract for report validation.

Methods

ValidationResult Validate(Report report);

Implementing Classes

- BaseValidator (abstract)
 - DroneValidator
 - SoldierValidator
 - RadarValidator
 - SignalValidator
-

Abstract Class: BaseValidator

Purpose

Provides common validation logic for all report types and eliminates code duplication.

Responsibility

- Validates common fields shared by all reports (Timestamp, Latitude, Longitude, Description)
- Defines template method pattern for validation workflow
- Delegates specific field validation to derived classes

Methods (Public)

public ValidationResult Validate(Report report)

Template method that validates both common and specific fields:

1. Calls `ValidateCommonFields(report)` - validates shared fields
2. If common validation passes, calls `ValidateSpecificFields(report)` - validates type-specific fields
3. Returns `ValidationResult` indicating success or failure

Methods (Protected)

protected `ValidationResult ValidateCommonFields(Report report)`

Validates common fields for all report types:

- **Timestamp:** Cannot be in the future, cannot be before 2020-01-01
- **Latitude:** Must be between 29.5000 and 33.5000
- **Longitude:** Must be between 34.0000 and 36.0000
- **Description:** Cannot be null/empty, must be 10-500 characters

Returns `ValidationResult.Failure(message)` if any validation fails, otherwise `ValidationResult.Success()`.

protected abstract `ValidationResult ValidateSpecificFields(Report report)`

Abstract method that derived validators must implement to validate type-specific fields.

Relationships

- **Implements:** `IValidator`
- **Inherited by:** `DroneValidator`, `SoldierValidator`, `RadarValidator`, `SignalValidator`

Class: ValidationResult

Purpose

Encapsulates the result of a validation operation.

Responsibility

- Indicates whether validation passed or failed
- Stores the reason for failure if applicable

Fields (Private)

- `private bool _isValid`
- `private string _errorMessage`

Properties (Public)

- `bool IsValid { get; }`
- `string ErrorMessage { get; }`

Constructors

`public ValidationResult(bool isValid, string errorMessage)`

Static Factory Methods

`public static ValidationResult Success()`

`public static ValidationResult Failure(string errorMessage)`

Relationships

- **Returned by:** All IValidator implementations
 - **Used by:** ReportPipeline
-

Class: DroneValidator

Purpose

Validates drone-specific report fields.

Responsibility

- Inherits from BaseValidator
- Validates only drone-specific fields (Altitude, ImageQuality)
- Common field validation is handled by BaseValidator

Methods (Protected)

`protected override ValidationResult ValidateSpecificFields(Report report)`

Validates:

- Report is DroneReport type
- Altitude: 100–10000
- ImageQuality: 1–100

Note: Common fields (Timestamp, Latitude, Longitude, Description) are validated by BaseValidator.ValidateCommonFields()

Relationships

- **Inherits from:** BaseValidator
 - **Validates:** DroneReport
-

Class: SoldierValidator

Purpose

Validates soldier-specific report fields.

Responsibility

- Inherits from BaseValidator
- Validates only soldier-specific fields (SoldierName, SoldierID, Unit, ConfidenceLevel)
- Common field validation is handled by BaseValidator

Methods (Protected)

protected override ValidationResult ValidateSpecificFields(Report report)

Validates:

- Report is SoldierReport type
- SoldierName: 2–50 characters
- SoldierID: exactly 7 digits
- Unit: 2–50 characters
- ConfidenceLevel: 1–5

Note: Common fields are validated by BaseValidator.ValidateCommonFields()

Relationships

- **Inherits from:** BaseValidator

- **Validates:** SoldierReport
-

Class: RadarValidator

Purpose

Validates radar-specific report fields.

Responsibility

- Inherits from BaseValidator
- Validates only radar-specific fields (Speed, Direction, Distance)
- Common field validation is handled by BaseValidator

Methods (Protected)

protected override ValidationResult ValidateSpecificFields(Report report)

Validates:

- Report is RadarReport type
- Speed: 0–2000
- Direction: 0–360
- Distance: 100–100000

Note: Common fields are validated by BaseValidator.ValidateCommonFields()

Relationships

- **Inherits from:** BaseValidator
 - **Validates:** RadarReport
-

Class: SignalValidator

Purpose

Validates signal-specific report fields.

Responsibility

- Inherits from BaseValidator
- Validates only signal-specific fields (Frequency, Content, Language, SignalStrength)
- Common field validation is handled by BaseValidator

Methods (Protected)

protected override ValidationResult ValidateSpecificFields(Report report)

Validates:

- Report is SignalReport type
- Frequency: 1.0–3000.0
- Content: 5–1000 characters
- Language: valid enum value
- SignalStrength: -120–0

Note: Common fields are validated by BaseValidator.ValidateCommonFields()

Relationships

- **Inherits from:** BaseValidator
 - **Validates:** SignalReport
-

Class: ReliabilityCalculator

Purpose

Centralized location for reliability score calculation logic.

Responsibility

- Delegates to each report's CalculateReliabilityScore method
- Ensures score is within valid range

Methods (Public)

public int Calculate(Report report)

Calls `report.CalculateReliabilityScore()` and ensures result is 1–10.

Relationships

- **Uses:** Report (polymorphically)
 - **Used by:** ReportPipeline
-

Class: PriorityCalculator

Purpose

Calculates report priority based on business rules.

Responsibility

- Analyzes report content and source type
- Applies priority determination rules
- Returns appropriate Priority enum value

Methods (Public)

public Priority Calculate(Report report)

Implements priority rules:

- Critical: missile/explosion/attack/fire keywords, Radar Speed ≥ 800 , Signal with target AND attack
- High: weapon/suspicious/border keywords, Drone Altitude < 500 , Radar Speed ≥ 400 , Soldier ConfidenceLevel ≥ 4 with movement
- Medium: movement/vehicle/activity keywords, Radar Speed ≥ 120 , ReliabilityScore ≥ 7
- Low: default

Private Helper Methods

private bool ContainsKeyword(string text, params string[] keywords)

Case-insensitive keyword search.

Relationships

- **Uses:** Report (polymorphically), DroneReport, RadarReport, SoldierReport, SignalReport

- **Used by:** ReportPipeline
-

Class: ClassificationCalculator

Purpose

Calculates security classification based on business rules.

Responsibility

- Analyzes report priority and content
- Applies classification determination rules
- Returns appropriate Classification enum value

Methods (Public)

public Classification Calculate(Report report)

Implements classification rules:

- TopSecret: Priority=Critical OR Content contains target/attack/missile
- Secret: Priority=High OR SourceType=Signal OR Description contains weapon/border
- Restricted: Priority=Medium OR SourceType=Soldier
- Unclassified: default

Private Helper Methods

private bool ContainsKeyword(string text, params string[] keywords)

Relationships

- **Uses:** Report (polymorphically), SignalReport, SoldierReport
 - **Used by:** ReportPipeline
-

Class: ReportRepository

Purpose

Stores and manages validated reports.

Responsibility

- Maintains collection of validated reports
- Provides search, filter, and retrieval operations
- Manages report status updates

Fields (Private)

- `private List<Report> _reports`

Constructor

```
public ReportRepository()
```

Initializes empty report list.

Methods (Public)

```
public void Add(Report report)
```

```
public List<Report> GetAll()
```

```
public List<Report> GetByStatus(ReportStatus status)
```

```
public List<Report> GetByPriority(Priority priority)
```

```
public List<Report> Search(string keyword)
```

```
public Report GetById(int reportId)
```

```
public void UpdateStatus(int reportId, ReportStatus newStatus)
```

```
public int GetTotalCount()
```

```
public int GetCountByStatus(ReportStatus status)
```

Relationships

- **Stores:** Report (and all derived types)
 - **Used by:** ReportPipeline, Program
-

Class: RejectedReportRepository

Purpose

Stores and manages rejected reports separately.

Responsibility

- Maintains collection of rejected reports
- Provides access to rejection reasons
- Enables analysis of validation failures

Fields (Private)

- `private List<Report> _rejectedReports`

Constructor

```
public RejectedReportRepository()
```

Methods (Public)

```
public void Add(Report report)
```

```
public List<Report> GetAll()
```

```
public int GetTotalCount()
```

```
public List<Report> GetByReason(string reasonKeyword)
```

Relationships

- **Stores:** Report (rejected instances)
- **Used by:** ReportPipeline, Program

Class: ReportPipeline

Purpose

Orchestrates the complete report processing workflow.

Responsibility

- Receives incoming reports
- Executes validation
- Calculates reliability, priority, classification
- Routes to appropriate repository
- Manages report lifecycle

Fields (Private)

- `private ReportRepository _validatedReports`
- `private RejectedReportRepository _rejectedReports`
- `private int _nextReportId`

Constructor

```
public ReportPipeline()
```

Initializes repositories and ID counter.

Methods (Public)

```
public void ProcessReport(Report report)
```

```
public ReportRepository GetValidatedReports()
```

```
public RejectedReportRepository GetRejectedReports()
```

```
public void DisplayStatistics()
```

Private Methods

```
private IValidator GetValidator(Report report)
```

```
private void ValidateReport(Report report)
```

```
private void CalculateMetrics(Report report)
```

```
private void StoreReport(Report report)
```

Processing Flow

1. Receive Report (Status = New)

2. Set Status = Validating
3. Get appropriate validator
4. Execute validation
5. If validation fails:
 - Set Status = Rejected
 - Set RejectionReason
 - Store in RejectedReportRepository
 - END
6. If validation succeeds:
 - Set Status = Validated
 - Calculate ReliabilityScore
 - Calculate Priority
 - Calculate Classification
 - Store in ReportRepository

Relationships

- **Uses:** Report, IValidator, DroneValidator, SoldierValidator, RadarValidator, SignalValidator, ReliabilityCalculator, PriorityCalculator, ClassificationCalculator, ReportRepository, RejectedReportRepository
 - **Used by:** Program
-

Class: Program

Purpose

Application entry point and user interface.

Responsibility

- Creates sample reports
- Demonstrates pipeline processing
- Displays results and statistics

Methods (Public/Static)

```
public static void Main(string[] args)
```

Private Helper Methods

```
private static void DisplayReport(Report report)
```

```
private static void DisplayValidatedReports(ReportRepository repository)
```

```
private static void DisplayRejectedReports(RejectedReportRepository repository)
```

Relationships

- **Uses:** All report types, ReportPipeline, ReportRepository, RejectedReportRepository
-

Object Communication Flow

Program

↓

Creates Report instances (DroneReport, SoldierReport, etc.)

↓

Passes to → ReportPipeline.ProcessReport()

↓

Status = New

↓

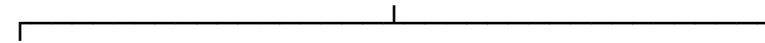
Status = Validating



GetValidator() → Returns appropriate IValidator



Validator.Validate() → Returns ValidationResult



IsValid = false



IsValid = true



Status = Rejected



Status = Validated



Set RejectionReason



ReliabilityCalculator.Calculate()



Store in



PriorityCalculator.Calculate()

RejectedReportRepository



ClassificationCalculator.Calculate()



Store in ReportRepository



Retrieves results



Displays to console

Relationships Summary

Inheritance Hierarchy

Report (abstract)

└─ DroneReport

└─ SoldierReport

└─ RadarReport

└─ SignalReport

Interface Implementation & Inheritance

IValidator (interface)

└─ BaseValidator (abstract)

└─ DroneValidator

└─ SoldierValidator

└─ RadarValidator

└─ SignalValidator

Composition

- ReportPipeline **contains** ReportRepository
- ReportPipeline **contains** RejectedReportRepository
- ReportRepository **contains** List
- RejectedReportRepository **contains** List

Dependencies

- ReportPipeline **depends on** all validators
- ReportPipeline **depends on** all calculators
- Validators **depend on** specific report types

- Calculators **depend on** Report (polymorphically)
- Program **depends on** ReportPipeline and all report types